An Empirical Analysis of Entangled Enhancement in an Autonomous Driving Pipeline of ML Components

***Abstract*—Entangled enhancement is a retraining that improves one component of a machine learning system (MLS) but fails to improve the whole system. This paper measures entangled enhancement in a driving pipeline whose perception, prediction and planning modules are machine learning (ML) components. The perception module is improved by 30 updates, while the prediction and planning modules stay fixed after training on the ground truth or on the output of the old perception module. When the prediction module was trained on the output of the old perception module, the prediction error increased in 23 of the 30 updates, in 11 by more than random variation and in 3 with statistical significance. When the prediction module was trained on the ground truth, the counts were 5, 0 and 0. The planning error changed by at most 0.13 m. No measure of output change that needs no ground truth predicted which updates harmed the prediction module.**

***Index Terms*—autonomous driving, entangled enhancement, machine learning system, model maintenance, retraining**

I. Introduction

An MLS is often built as a pipeline, which is a set of ML components connected in a line, where each component uses the output of the preceding component. A component that comes earlier in the line is upstream of a component that comes later, and the later component is downstream. Retraining an upstream component can improve that component and still fail to improve the whole system. Wang and Machida call this effect entangled enhancement and model when an MLS should retrain each component [1], [2]. The models need a probability of harm that has not been measured on a real system.

A driving pipeline has three components. The perception module finds the objects around the ego vehicle, which is the vehicle that the pipeline drives. The prediction module predicts where each object will move, and the planning module chooses the path of the ego vehicle. An earlier experiment in this study retrained the perception module of such a pipeline 15 times. In 6 of the 15 updates, the planned path moved further from the path of the human driver. The prediction module and the planning module in that pipeline were fixed rules. Fixed rules cannot adapt to the mistakes of the perception module. That experiment therefore could not test the adaptation that the theory of entangled enhancement describes.

This paper replaces both fixed rules with ML components, trained once on the ground truth, the object positions labelled by hand, and once on the output of the old perception module. Only the second way allows adaptation to the old mistakes. The comparison of the two ways therefore shows whether a harm comes from that adaptation.

This paper makes three contributions.

1) A driving pipeline of three ML components, trained on a reduced version of the nuScenes dataset [3] and checked against simple baselines.
2) A series of 30 updates of the perception module, with the number of harmful updates counted at three levels of evidence.
3) A test of nine measures of output change that need no ground truth, as a possible symptom of entangled enhancement.

II. Background and Related Work

This section summarises the earlier work that the experiment builds on. Sculley et al. [4] describe how a change to one component of an MLS changes the behaviour of every component that uses the changed output. Wu et al. [5] show that improving one model of a system can make the whole system worse. A harm of this kind can have two causes. In the first cause, the downstream components adapted during training to the mistakes of the old upstream component and are disturbed when those mistakes change. In the second cause, the upstream component improves on average but becomes worse on the cases that matter downstream.

AutoBot-Ego [6] predicts the path of an object, and PlanT [7] plans the path of the ego vehicle from a list of objects. Weng et al. [8] show that a prediction module can learn by attention which of the objects found in different images are the same object. The prediction error increases when the input comes from a real perception module instead of the ground truth [9]. A planning module that receives the state of the ego vehicle can score well on nuScenes while ignoring the objects [10], [11].

Rabanser et al. [12] detect a change in the data seen by a model without ground truth from distances between distributions, where a distribution is the pattern of values that an output takes over many inputs. This paper applies such distances to the outputs of the modules as a candidate symptom.

III. Methodology

This section defines the notation, states the two hypotheses, and describes the workflow and the update procedure that test the hypotheses.

A. Pipeline and Notation

The perception module C_1 finds the objects in the image of the front camera. The prediction module C_2 predicts the path of each object over the next 3 s, and the planning module C_3 chooses the path of the ego vehicle over the same 3 s. An update is one retraining of C_1 on data from a new city. We call C_1 before an update the old C_1 and C_1 after the update the new C_1.

We write δ_1 for the gain of C_1 in mAP50, the standard accuracy measure for object detection. The prediction error is the minADE, which is the average distance between the predicted path and the true path of an object, taken for the best of the paths that C_2 predicts. The planning error is the distance between the planned position and the position of the human driver at 3 s. We write Δ_2 and Δ_3 for the increase of the prediction error and of the planning error after an update. An update shows entangled enhancement when

$$\delta_1 > 0 \quad \text{and} \quad (\Delta_2 > 0 \text{ or } \Delta_3 > 0). \tag{1}$$

B. Hypotheses

H1 (existence): An update of C_1 can increase the prediction error or the planning error of a pipeline of three ML components, even when the update makes C_1 more accurate. The increase is larger when C_2 and C_3 were trained on the output of the old C_1 than when C_2 and C_3 were trained on the ground truth.

H2 (symptom): The change in the output of C_1 and C_2 after an update, measured without ground truth, is larger for the updates that meet condition (1) than for the other updates. The gain δ_1 alone does not separate the two groups.

C. Workflow

Fig. 1 shows the six stages of the experiment. The data are split into four parts. Boston 1 and Boston 2 train the modules, Singapore 1 provides the data for the updates, and Singapore 2 is the test set. The old C_1 is trained with two random seeds, where a random seed is the starting number for the random choices made during training. C_2 and C_3 are trained once in two ways. Isolated training uses the ground truth, and old-pipeline training uses the output of the old C_1. Each of the 30 updates is then tested in the full pipeline with C_2 and C_3 kept fixed.

D. Update Procedure

Algorithm 1 gives the procedure for one old C_1, where iso marks isolated training and op marks old-pipeline training. The random variation τ_2 and τ_3 is the largest change of the errors when the old C_1 is replaced by the other old C_1, which differs only in the random seed. The first level of evidence counts any increase, and the second level counts an increase above the random variation. The third level counts an increase whose 95 % bootstrap interval lies above zero. The bootstrap repeats the measurement on test sets drawn with replacement from the test scenes.

Algorithm 1: Update procedure for one old C_1

Input : old C_1^{old}, settings $\mathcal{S}$, Boston 2, Singapore 1, Singapore 2
Output: δ_1, Δ_2, Δ_3 and evidence level per update and training
Train $C_2^{\text{iso}}, C_3^{\text{iso}}$ on the ground truth of Boston 2
Train $C_2^{\text{op}}, C_3^{\text{op}}$ on the output of C_1^{old} on Boston 2
foreach *training* $r \in \{\text{iso}, \text{op}\}$ **do**
  $E_2^r, E_3^r \leftarrow$ errors of $C_1^{\text{old}} \to C_2^r \to C_3^r$ on Singapore 2
$\tau_2, \tau_3 \leftarrow$ random variation from the other old C_1
foreach *setting* $s \in \mathcal{S}$ **do**
  $C_1^{\text{new}} \leftarrow$ retrain C_1^{old} on Singapore 1 with s
  $\delta_1 \leftarrow \text{mAP50}(C_1^{\text{new}}) - \text{mAP50}(C_1^{\text{old}})$
  foreach *training* $r \in \{\text{iso}, \text{op}\}$ **do**
    $\Delta_2, \Delta_3 \leftarrow$ errors of $C_1^{\text{new}} \to C_2^r \to C_3^r$ minus E_2^r, E_3^r
    record the three levels of evidence
  record the output change of C_1 and C_2 for H2

IV. Experimental Setup

This section gives the data, the updates of C_1, the ML models for C_2 and C_3, and the measurements. All models were trained on one Tesla T4 graphics card.

A. Data

The experiment uses 350 of the 850 scenes of nuScenes [3], which are driving clips of about 20 s from Boston and Singapore, with the full-size images of the front camera only (4.6 GB). Boston 1 (100 scenes) trains C_1, and Boston 2 (100 scenes) trains C_2 and C_3, with 14 of the scenes kept for validation. Separate scenes give C_2 and C_3 the realistic mistakes of C_1 [9]. Singapore 1 (100 scenes) is used for the updates, and Singapore 2 (50 scenes, 2,020 images) is used for testing. A keyframe is an image with labelled objects, recorded twice per second. The test set has 1,720 test samples, one for every keyframe with 3 s of future.

B. Updates of the Perception Module

C_1 is YOLOv11n [13], trained for 10 passes over Boston 1 with the random seeds 0 and 1. The two old C_1 reach an mAP50 of 0.109 and 0.117 on Singapore 2. An update retrains an old C_1 on Singapore 1 with one of 15 settings. Nine settings use 100 %, 50 % or 25 % of the images with 10 passes, and six use all images with 5 or 20 passes, each with the seeds 1, 2 and 3. The 15 settings applied to both old C_1 give 30 updates. The test scenes are never used to choose a model.

C. ML Prediction and Planning Modules

C_2 follows AutoBot-Ego [6]. For each current object, a transformer reads the objects of the last four keyframes relative to that object and outputs six possible paths of 3 s, without a hand-written rule that links objects across images [8]. C_3 follows PlanT [7]. Every object is one input with the current position, the most likely path from C_2, the class and the confidence. Two further inputs hold the driving command and the path of the ego vehicle over the last 1.5 s. The command is left, straight or right [10], and a recurrent decoder outputs six points of the planned path.

Both models were trained on Boston 2 with early stopping on the validation scenes and with mirror and rotation

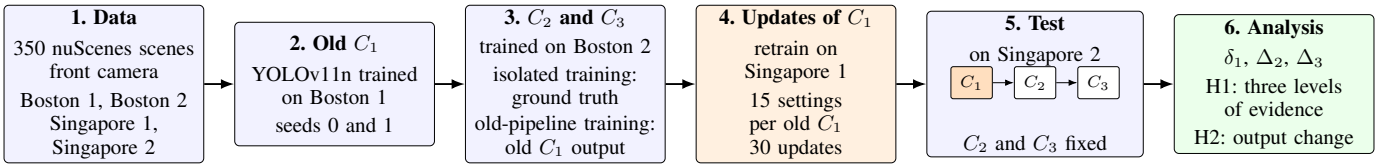


Fig. 1. Workflow of the experiment

TABLE I
ML COMPONENTS AGAINST SIMPLE BASELINES ON THE TEST SET (M)

Model and input	Model	Baseline
C_2 on the ground truth	0.77	4.48 (stays still)
C_2 on the output of C_1	2.39 to 2.95	5.39 to 5.85 (stays still)
C_3 without ego input	9.3 to 12.5	9.25 (average path)
C_3 with the ego path	3.70 to 4.16	9.25 (average path)

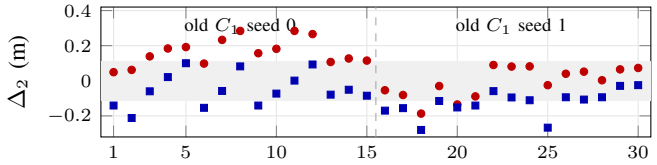


Fig. 2. Δ_2 in the 30 updates. Red circles show old-pipeline training, blue squares show isolated training, and the grey band shows the random variation.

augmentation. An input of the ego path can let a planning module ignore the objects [11], and the experiment checks whether C_3 uses the objects. Removing the objects from the input of C_3 moves the planned position at 3 s by 0.73 to 1.65 m on average, which is above the threshold of 0.5 m fixed before the experiment.

D. Training and Measurements

C_2 and C_3 are trained once with isolated training and once with old-pipeline training for each old C_1, and the trained models stay fixed during all updates. The prediction error is computed over the objects found by C_1 that are matched one to one with a ground truth object of the same class. The collision rate is the fraction of test samples in which the planned path overlaps an object. The outputs of C_2 and C_3 on the ground truth stayed exactly the same before and after every update, which confirms that only the output of C_1 changed. The random variation is $\tau_2 = 0.112\,\text{m}$ and $\tau_3 = 0.018\,\text{m}$. The bootstrap draws the 50 test scenes 2,000 times with replacement.

V. Results and Discussion

This section first checks that C_2 and C_3 work, because a count of harmful updates is meaningful only for working modules, and then reports and discusses the results. Every update made C_1 more accurate, with δ_1 between 0.048 and 0.118.

A. Validity of the ML Components

Table I compares C_2 and C_3 with simple baselines on the test set. C_2 beats the baseline in which every object stays still. The variant of C_3 without any ego input does not beat the baseline that drives the average Boston path for each command. The experiment therefore uses the variant with the ego path, which has a collision rate of 0.05 to 0.08 against 0.18 for the baseline.

TABLE II
UPDATES, OUT OF 30, WITH AN INCREASE OF THE ERROR

	Prediction Δ_2		Planning Δ_3	
Level of evidence	**op**	**iso**	**op**	**iso**
Any increase	23	5	18	29
Above random variation	11	0	13	29
Bootstrap interval above zero	3	0	1	10

B. Prediction Module

Fig. 2 shows Δ_2 for every update and both ways of training. In all 30 updates, Δ_2 after old-pipeline training is larger than Δ_2 after isolated training. The difference is largest for the updates from the old C_1 with seed 0, where Δ_2 after old-pipeline training lies between $+0.049$ and $+0.284\,\text{m}$ with a mean of $+0.165\,\text{m}$.

Table II counts the updates with an increase at the three levels of evidence. Above random variation, the prediction error increased in 11 updates after old-pipeline training and in none after isolated training. All 11 updates start from the old C_1 with seed 0. With 50 test scenes, the bootstrap interval of Δ_2 spans about $\pm 0.25\,\text{m}$. Only 3 updates after old-pipeline training and none after isolated training have an interval above zero.

The comparison points to the first cause of entangled enhancement. Both versions of C_2 receive the same output of the same new C_1, and only the version that learned from the old C_1 becomes worse. The C_2 trained on the old C_1 with seed 0 also becomes worse by 0.112 m when the old C_1 is replaced by the other old C_1, although the two old C_1 differ by only 0.009 in mAP50. This C_2 reacts to a change in the mistakes of C_1 and not to a change in the accuracy of C_1. The C_2 trained on the old C_1 with seed 1 improves by 0.098 m under the same replacement, and no update from seed 1 harms this C_2 by more than random variation. The harm after an update is larger for the old C_1 with the stronger adaptation.

C. Planning Module

The planning error changed little. The mean Δ_3 lies between -0.006 and $+0.058$ m, and the largest value is 0.13 m against a planning error of about 4 m. The increase appears more often after isolated training than after old-pipeline training at every level in Table II. Adaptation to the mistakes of the old C_1 cannot explain an increase after isolated training.

The small size of Δ_3 has a direct reason. C_3 plans mainly from the path of the ego vehicle, and the objects move the planned position at 3 s by only 0.73 to 1.65 m. The more frequent increase after isolated training points to the second cause. The C_3 trained on the ground truth meets the predicted paths of C_2 only at test time, and a change in the objects that the new C_1 finds can affect this C_3 more than the C_3 trained on the output of the old C_1.

D. New-Pipeline Training

Retraining C_2 and C_3 with C_1 can remove the harm. New-pipeline training retrains C_2 and C_3 on the output of the new C_1, and was applied to two further updates, one from each old C_1 with all images and 10 passes. After old-pipeline training, the prediction error after the update is 2.75 and 2.61 m, and the planning error is 4.14 and 4.00 m. After new-pipeline training, the prediction error is 2.39 and 2.45 m, and the planning error is 3.71 and 3.70 m.

New-pipeline training lowers the prediction error by 0.36 and 0.16 m and the planning error by 0.44 and 0.30 m. The errors also fall below the errors before the update (2.62 and 2.66 m, 4.16 and 4.00 m), and the update becomes an improvement of the whole pipeline. This result supports the retraining of downstream components that Wang and Machida model [1], [2]. A symptom of harm is useful because that retraining has a cost.

E. Symptom of Entangled Enhancement

Nine measures of output change need no ground truth, seven on the output of C_1 and two on the output of C_2. The rank correlation with Δ_2 is close to 1 when a larger measure always comes with a larger Δ_2. The AUROC is the probability that a harmful update ($\Delta_2 > 0$) has a larger measure than a harmless update. Over all 30 updates, the shift of the predicted position of the same object has the highest rank correlation ($+0.68$, AUROC 0.81). For each old C_1 separately, the rank correlation is only $+0.29$ and -0.28. The high value over all updates comes from the difference between the two old C_1. The gain δ_1 has a rank correlation of $+0.04$. No measure separates harmful and harmless updates consistently, and H2 is not supported.

The rank correlation of δ_1 shows that the accuracy gain of C_1 says nothing about the harm to C_2. The measures on the output of C_1 describe the update alone, and the harm depends on how strongly C_2 adapted to the old C_1. The random variation in Section V-B gives a hint for a better symptom. The old C_1 whose C_2 was disturbed by the replacement of the old C_1 is also the old C_1 whose updates harmed C_2. A measure of how strongly C_2 reacts to any change of the output of C_1 can therefore describe the adaptation of C_2 before an update is made.

VI. Conclusion

In 30 updates of the perception module, the prediction module became worse more often and by more after old-pipeline training than after isolated training. This result supports H1 for the prediction module and points to adaptation to the mistakes of the old perception module as the cause. The planning error changed little. No measure of output change that needs no ground truth predicted the harm, and H2 is not supported. Work in progress develops a method for the early detection of entangled enhancement, which flags a harmful update of the perception module before the update is used and before ground truth for the new data exists.

References

[1] Z. Wang and F. Machida, "Maintaining performance of a machine learning system against imperfect retraining," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.

[2] Z. Wang and F. Machida, "Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems," in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.

[3] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.

[4] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.

[5] R. Wu, C. Guo, A. Hannun, and L. van der Maaten, "Fixes that fail: Self-defeating improvements in machine-learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 11 745–11 756.

[6] R. Girgis, F. Golemo, F. Codevilla, M. Weiss, J. A. D'Souza, S. E. Kahou, F. Heide, and C. Pal, "Latent variable sequential set transformers for joint multi-agent motion prediction," in *International Conference on Learning Representations (ICLR)*, 2022.

[7] K. Renz, K. Chitta, O.-B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger, "PlanT: Explainable planning transformers via object-level representations," in *Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 205, 2022.

[8] X. Weng, B. Ivanovic, K. Kitani, and M. Pavone, "Whose track is it anyway? Improving robustness to tracking errors with affinity-based trajectory prediction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[9] Y. Xu, L. Chambon, É. Zablocki, M. Chen, A. Alahi, M. Cord, and P. Pérez, "Towards motion forecasting with real-world perception inputs: Are end-to-end approaches competitive?" in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 18 428–18 435.

[10] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuScenes," *arXiv preprint arXiv:2305.10430*, 2023.

[11] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez, "Is ego status all you need for open-loop end-to-end autonomous driving?" in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[12] S. Rabanser, S. Günnemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[13] G. Jocher and J. Qiu, "Ultralytics YOLO11," https://github.com/ultralytics/ultralytics, 2024.